INTERMEDIATE CONSTRUCTIVE (math-based)

```
// Build a valid answer directly using a pattern — parity, modular
// structure, or a clever formula — instead of searching. The skill
// is spotting the right observation, not memorizing code.
```

Common Constructive Observations

```
Odd / Even (Parity)  — does the answer depend on n being odd or even?
Prefix / Suffix  — can you build left-to-right or right-to-left?
Reverse  — does reversing the array/string reveal a pattern?
Rotation  — does shifting elements cyclically help?
Alternating Pattern  — +1/-1, high/low, or 0/1 repeating structure
Cyclic Assignment  — assign values in a repeating 1..k cycle
Greedy Construction  — always take the locally valid choice
Smallest/Largest Valid Answer  — construct the extreme case directly
Work Backwards  — start from the target/end state, build toward input
```

Demo 1: Alternating pattern construction

```
vector<int> a(n); a[0] = 0;
for (int i=1;i<n;i++) a[i] = a[i-1] + (i%2==0 ? 1 : -1);
```

Demo 2: Cyclic assignment (repeating pattern of k values)

```
vector<int> a(n);
for (int i=0;i<n;i++) a[i] = (i % k) + 1;
```

Demo 3: Greedy digit-by-digit — smallest n-digit number summing to s

```
string result = ""; int remaining = s;
for (int i=0;i<n;i++) {
    int digitsLeft = n - i - 1;
    int d = max(0, remaining - digitsLeft*9);  // smallest digit still completable
    d = max(i==0 ? 1 : 0, d);  // no leading zero
    result += ('0' + d);
    remaining -= d;
}
// Before coding: try small cases (n=1,2,3) by hand and look for the
// pattern — most constructive problems fit one of the categories above.

// Demo 4: Work backwards — reconstruct array from given prefix XOR/sum
// (common pattern: you're given the "result" array, must recover original)
vector<int> pre = {0, 3, 1, 6};   // given prefix XOR values, pre[0]=0
vector<int> a(pre.size() - 1);
for (int i = 0; i < a.size(); i++)
    a[i] = pre[i] ^ pre[i+1];      // recover original element from consecutive
prefix values

// Demo 5: Parity-based pairing — pair smallest with largest (common constructive
trick)
sort(a.begin(), a.end());
vector<int> result;
int l = 0, r = n - 1;
while (l <= r) {
    result.push_back(a[l++]);
    if (l <= r) result.push_back(a[r--]);
}
```